

JSON

CC-BY: Maria Skeppstedt

1. The json format

What is the json format?

From Wikipedia: "JSON is a language-independent data format. It was derived from JavaScript, but many modern programming languages include code to generate and parse JSON-format data. JSON filenames use the extension .json"

The json module

<https://docs.python.org/3/library/json.html>
json — JSON encoder and decoder

Source code: [Lib/json/__init__.py](#)

JSON (JavaScript Object Notation), specified by [RFC 7159](#) (which obsoletes [RFC 4627](#)) and by [ECMA-404](#), is a lightweight data interchange format inspired by [JavaScript](#) object literal syntax (although it is not a strict subset of JavaScript [\[1\]](#)).

Warning: Be cautious when parsing JSON data from untrusted sources. A malicious JSON string may cause the decoder to consume considerable CPU and memory resources. Limiting the size of data to be parsed is recommended.

`json` exposes an API familiar to users of the standard library `marshal` and `pickle` modules.

Encoding basic Python object hierarchies:

```
>>> import json
>>> json.dumps(['foo', {'bar': ('baz', None, 1.0, 2)}])
'["foo", {"bar": ["baz", null, 1.0, 2]}]'
```

```
>>>
```

Advantages with json

Compared to e.g., pickle, there are some advantages with using the json format. Eg.

- ▶ The data is human-readable
- ▶ The format is not Python-specific. Therefore, it can be read and written by other programming languages

2. Saving and loading a dictionary in json format

Saving a dictionary in json format

A dictionary can be saved in json format.
You open the file text mode.

```
1 import json
2
3 file_name = "animal-words.json"
4
5 # Dump
6 with open(file_name, "w") as fp:
7     json.dump({"brun" : ["brown"],\
8                 "kanin": ["rabbit", "bunny"]}, fp)
9
10 # Load
11 with open(file_name, "r") as read_from:
12     swedish_animal_words = json.load(read_from)
13     print(swedish_animal_words)
```

'brun': ['brown'], 'kanin': ['rabbit', 'bunny']

What does the json file look like?

The file "animal-words.json", looks like this:

```
{"brun": ["brown"], "kanin": ["rabbit", "bunny"]}
```


Indenting the json output

You can indent the resulting json, to make it easier to read.

```
1 import json
2 file_name = "animal-words.json"
3
4 with open(file_name, "w") as fp:
5     json.dump({"brun" : ["brown"],\
6                 "kanin": ["rabbit", "bunny"]}, fp, indent=8)
```

Results in a file that looks like this:

```
{
    "brun": [
        "brown"
    ],
    "kanin": [
        "rabbit",
        "bunny"
    ]
}
```

3. Other Python types in json format

(other types than dictionaries)

Convert to json

The following Python types can be converted into json:

Python	JSON
dict	object
list, tuple	array
str	string
int, float, int- & float-derived Enums	number
True	true
False	false
None	null

Examples

```
1 example = { "list" : [1, 2],
2     "tuple": (3, 4),
3     "str": "Hello world",
4     "int": 1,
5     "float": 1.0,
6     "True": True,
7     "False": False,
8     "None": None}
9
10 with open("example.json", "w") as fp:
11     json.dump(example, fp, indent=4)
```

Example from previous slide

results in the following content in 'example.json'

```
{
  "list": [
    1,
    2
  ],
  "tuple": [
    3,
    4
  ],
  "str": "Hello world",
  "int": 1,
  "float": 1.0,
  "True": true,
  "False": false,
  "None": null
}
```

Loading the json file

from previous slide

Loading the json file from previous slide results in the original Python dictionary, with Python types.

(Except that the tuple has become a list, since json doesn't make any distinction between list and tuple).

(Also, note that in json, "None" is represented as "null", and "True"/"False" with a lower case first character, but they are converted back to Python style.)

```
1 with open("example.json", "r") as read_from :  
2     print(json.load(read_from))
```

```
{'list': [1, 2], 'tuple': [3, 4], 'str': 'Hello world', 'int': 1, 'float': 1.0,  
'True': True, 'False': False, 'None': None}
```

4. dump/dumps and load/loads

dumps

Converts to a json string

```
1 import json
2
3 example = { "list" : [1, 2], "str": "Hello world",
4             "float": 1.0, "False": False, "None": None}
5
6 s = json.dumps(example, indent=4)
7 print(s)
```

```
{
    "list": [
        1,
        2
    ],
    "str": "Hello world",
    "float": 1.0,
    "False": false,
    "None": null
}
```


dumps

Converts to a json string

The type of s is a string:

```
1 import json
2
3 example = { "list" : [1, 2], "str": "Hello world",
4             "float": 1.0, "False": False, "None": None}
5
6 s = json.dumps(example, indent=4)
7 print(type(s))
```

<class 'str'>

loads

Converts from a json string

```
1 import json
2
3 example = { "list" : [1, 2], "str": "Hello world",
4             "float": 1.0, "False": False, "None": None}
5 s = json.dumps(example, indent=4)
6
7 loaded = json.loads(s)
8 print(loaded)
```

```
{'list': [1, 2], 'str': 'Hello world',
'float': 1.0, 'False': False, 'None': None}
```

5. json databases

What is a json database?

A json database (not covered in this course) is a database that stores the data in json format. That makes it more flexible than the row-columns format of the SQL database. An example using PyMongo code for the document database MongoDB:

```
1  ...
2  collection = db['example']
3
4  # Insert one document
5  document = {'name': 'Lisa', 'city': 'Stockholm'}
6  collection.insert_one(document)
7
8  # Insert many documents at once
9  documents = [
10     {'name': 'Ewa', 'city': 'Warsaw'},
11     {'name': 'Herbert', 'city': 'Berlin'}
12 ]
13 collection.insert_many(documents)
```
